

Busting .NET Myths

Jason Bock
Staff Software Engineer
Rocket Mortgage

PRAIRIE DEV CON

WEB | DEV | CLOUD | AI



Personal Info

- <https://jasonbock.net>
- <https://bsky.app/profile/jasonbock.net>
- <https://github.com/jasonbock>
- <https://youtube.com/JasonBock>
- <https://www.linkedin.com/in/jasonrbock/>
- jason.r.bock@outlook.com

Downloads

<https://github.com/JasonBock/BustingDotNetMyths>

<https://github.com/JasonBock/Presentations>

Overview

- Goals
- Myths
- Summary

Remember...

<https://github.com/JasonBock/BustingDotNetMyths>

<https://github.com/JasonBock/Presentations>

Goals

Busting .NET Myths



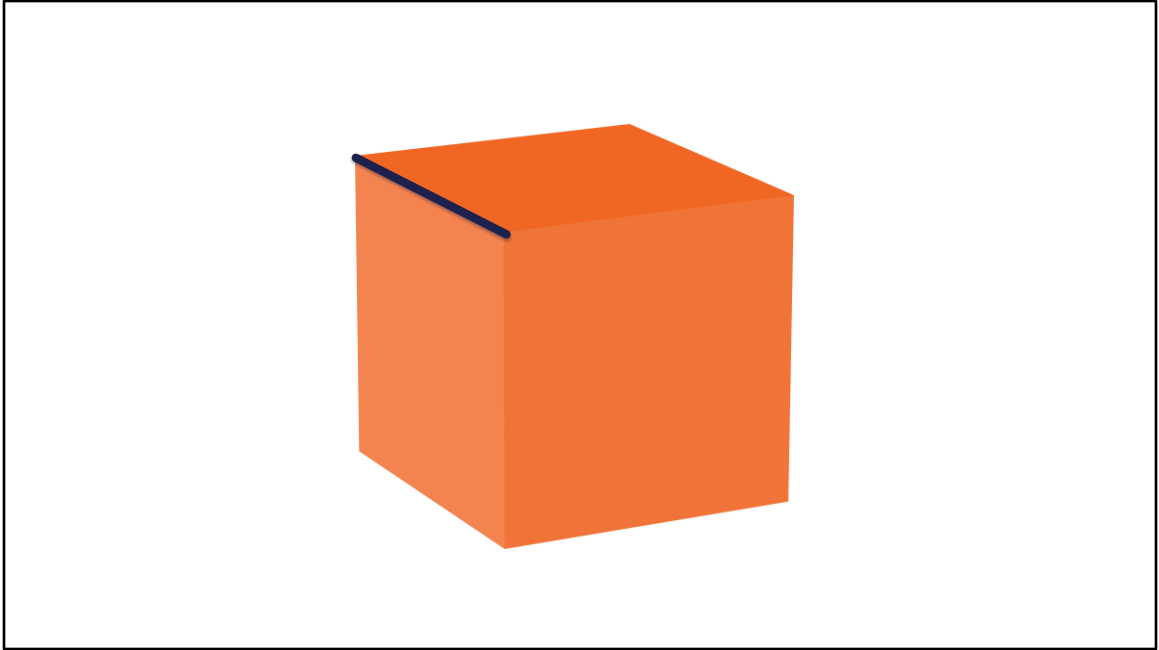
This talk is partially about having fun. I mean, EVERY talk should be engaging, but what I mean by “fun” is ...



... that it should be exploratory. It's time to dig around in the framework and see how things really work compared to some views and statements that I've heard and read over the years as a .NET developer. Granted, it can be dark and scary and somewhat intimidating to visit places that few have gone before, but the end result can be rewarding.



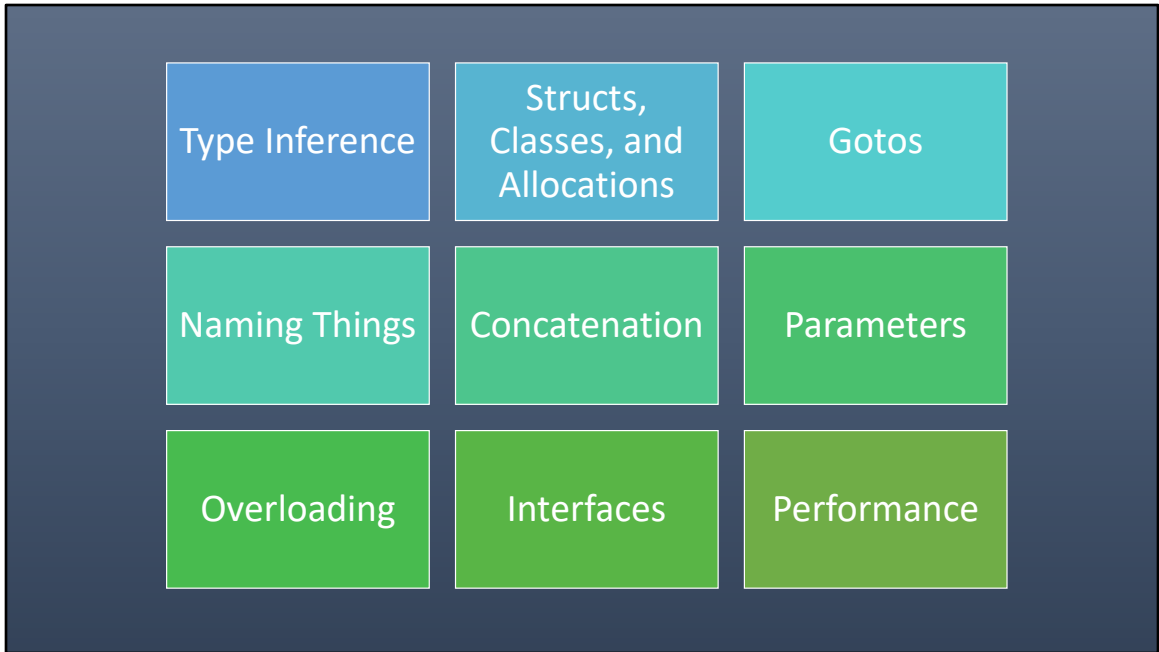
There are a couple of reasons why myths and untrue statements exist. One is change. Things change in languages, engines, packages, etc. and if you're not aware of these deltas, you may think that something works in a certain way when it doesn't.



Another are edge cases. Granted, we're going to do some exploring, but there are a couple of "myths" that, arguably, you'll probably never run into with the vast majority of programs you write. But knowledge is power, right?



And some are just big piles of crap 😊. For whatever reason, people will spread false information, and unfortunately they can permeate into people's internal knowledge base. We won't cover a lot of these, but we will address a couple of them



Here are the topics we're going to address.

Myth: Type Inference

Busting .NET Myths

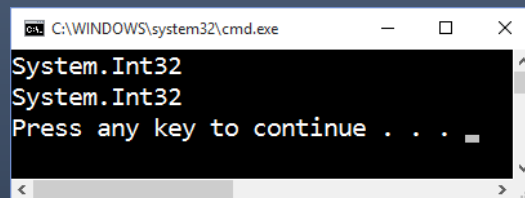
Myth: The “var” keyword makes your variables dynamic



We'll start out simple

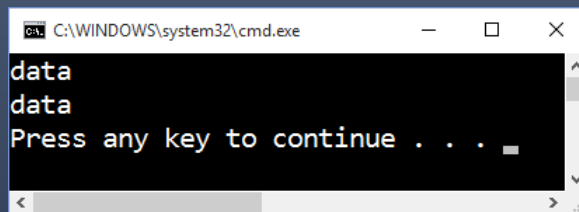
Myth: I still hear people say this about “var”. Nothing could be further from the truth. This isn't Clojure, Ruby or Python (though those languages are cool too).

```
int xExplicit = 41;  
Console.Out.WriteLine(xExplicit.GetType().FullName);  
  
var xInferred = 44;  
Console.Out.WriteLine(xInferred.GetType().FullName);
```



It's very simple. All "var" does is infer the type for you. In this case, the output is exactly the same. "xInferred" is an int; you can't change it to a string later on.

```
Dictionary<int, string> pairsExplicit =  
    new Dictionary<int, string> { { 22, "data" } };  
Console.Out.WriteLine(pairsExplicit[22]);  
  
var pairsInferred =  
    new Dictionary<int, string> { { 22, "data" } };  
Console.Out.WriteLine(pairsInferred[22]);
```



Where “var” shines is with long type names, like, anything longer than 3 characters 😊


```
public interface IBase
{
    void DoSomething();
}

public class ABase : IBase
{
    public void DoSomething() { }
}

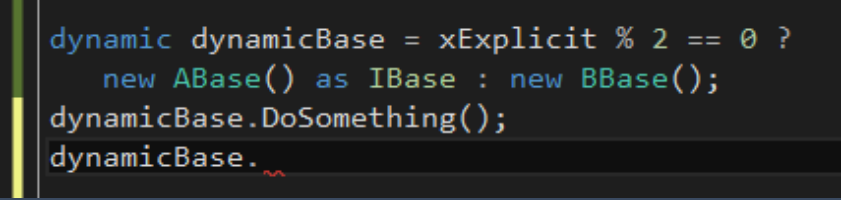
public class BBase : IBase
{
    public void DoSomething() { }
}
```

Nothing is perfect though. Let's say I had the following inheritance structure.

```
var varBase = xExplicit % 2 == 0 ?  
    new ABase() as IBase : new BBase();  
varBase.DoSomething();  
  
IBase varBaseViaIf = null;  
  
if (xExplicit % 2 == 0)  
{  
    varBaseViaIf = new ABase();  
}  
else  
{  
    varBaseViaIf = new BBase();  
}  
  
varBaseViaIf.DoSomething();
```

Note that even though both types inherit from IBase, the ternary operator doesn't know that. So you have to cast the first type. That, or use an if statement.

```
dynamic dynamicBase = xExplicit % 2 == 0 ?  
    new ABase() as IBase : new BBase();  
dynamicBase.DoSomething();
```



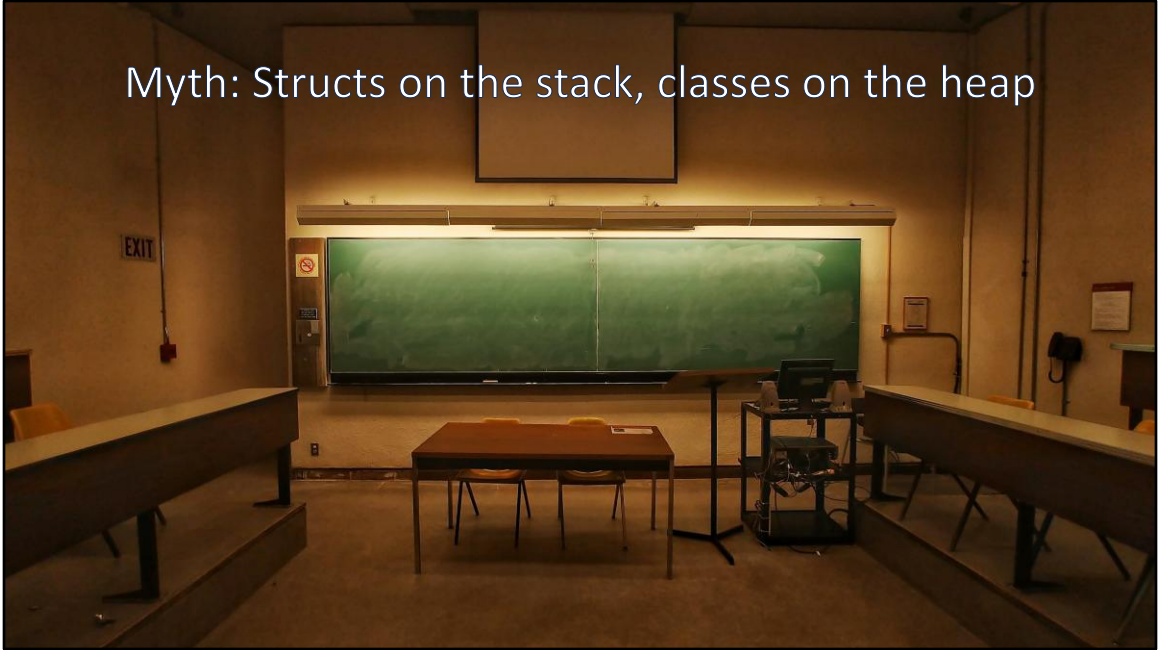
```
dynamic dynamicBase = xExplicit % 2 == 0 ?  
    new ABase() as IBase : new BBase();  
dynamicBase.DoSomething();  
dynamicBase.
```

Even with dynamic, I still need the cast. And when you use dynamic, you get no Intellisense.

Myth: Structs, Classes, and Allocations

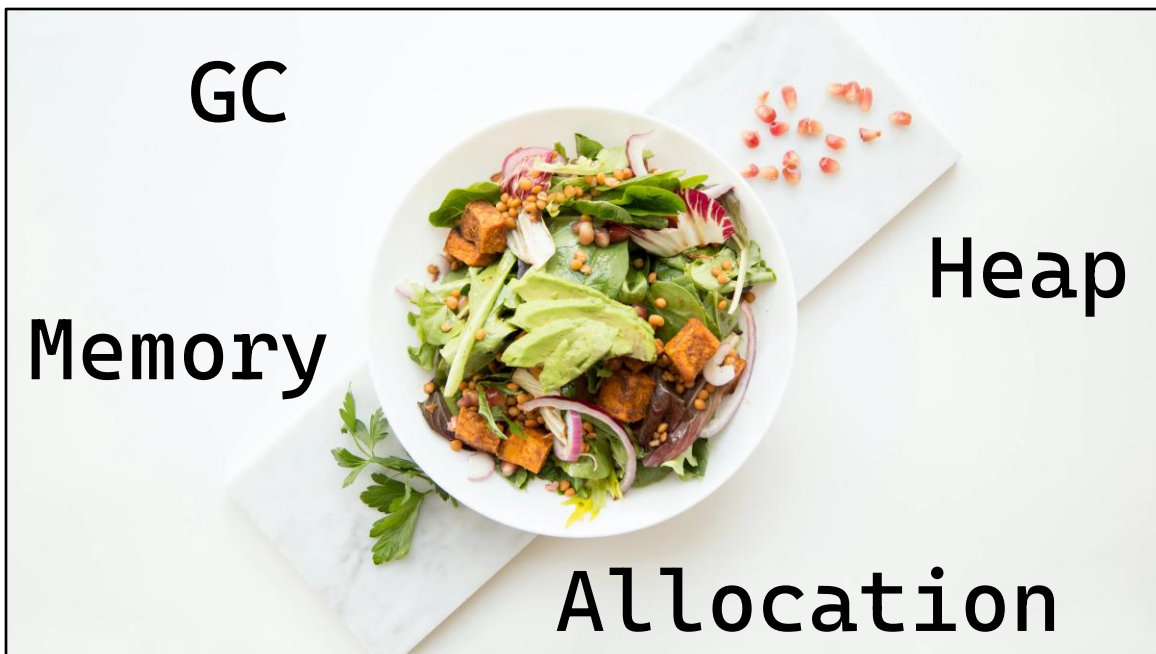
Busting .NET Myths

Myth: Structs on the stack, classes on the heap



Myth: This is a common one. If you make a struct, an instance of it always goes on the stack, class instances go on the heap. This comes up a lot (or historically it did) in interview scenarios as kind of a “prove your intelligence” about something that, while important to know, it’s not as straightforward as one may make it out to be.

<https://unsplash.com/photos/green-chalkboard-FhLiYoqHhvA>



We could start getting into a whole word salad of technical terms, but the key part of this debate is the design-time semantics of reference and value types. Where they are stored is completely immaterial.

https://unsplash.com/photos/cooked-food-with-sliced-vegetables-in-white-bowl-OzBLe_Eg1mg

```
public sealed class ReferenceData
{
    public ReferenceData(int data)
    {
        this.Data = data;
    }

    public void Increment()
    {
        this.Data++;
    }

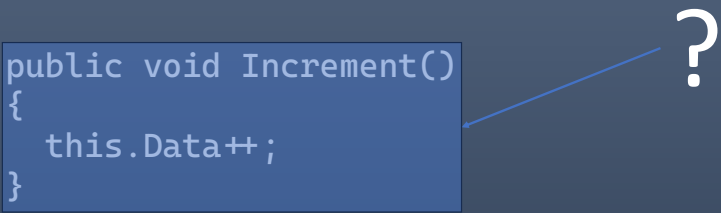
    public int Data { get; private set; }
}
```

Here's a quick example. This is a reference type.

```
public struct ValueData
{
    public ValueData(int data)
    {
        this.Data = data;
    }

    public void Increment()
    {
        this.Data++;
    }

    public int Data { get; private set; }
}
```



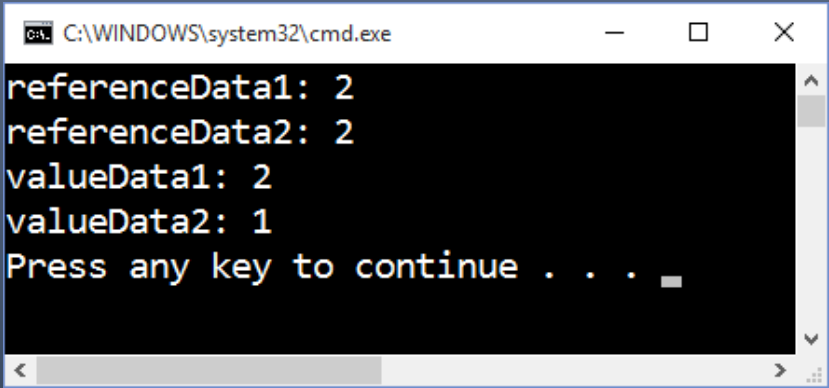
Here's a value type. Both are the same except for the "struct" and "class" definitions. Usually value types are immutable, but I'm doing this on purpose to prove a point.


```
var referenceData1 = new ReferenceData(1);
var referenceData2 = referenceData1;
referenceData1.Increment();

var valueData1 = new ValueData(1);
var valueData2 = valueData1;
valueData1.Increment();

Console.Out.WriteLine(
    $"{nameof(referenceData1)}: {referenceData1.Data}");
Console.Out.WriteLine(
    $"{nameof(referenceData2)}: {referenceData2.Data}");
Console.Out.WriteLine(
    $"{nameof(valueData1)}: {valueData1.Data}");
Console.Out.WriteLine(
    $"{nameof(valueData2)}: {valueData2.Data}");
```

Now, what will this do?



```
C:\WINDOWS\system32\cmd.exe  
referenceData1: 2  
referenceData2: 2  
valueData1: 2  
valueData2: 1  
Press any key to continue . . .
```

The value type is copied. The reference type is referenced. That's it.

“...in the Microsoft implementation of C# on the desktop CLR, value types are stored on the stack when the value is a local variable or temporary that is not a closed-over local variable of a lambda or anonymous method, and the method body is not an iterator block, and the jitter chooses to not enregister the value.”

This is a more precise definition. In some cases even a struct (or value type) isn't on the stack, though it's pretty rare.

<https://blogs.msdn.microsoft.com/ericlippert/2010/09/30/the-truth-about-value-types/>

https://unsplash.com/photos/white-brick-wall-4Zaq5xY5M_c

“Finally, the CLR does allow ‘ref return types’; you could in theory have a method ‘ref int M() { ... }’ that returned a reference to an integer variable. **If for some bizarre reason we ever decided to allow that in C#,** we’d have to fix up the compiler and verifier so that they ensured that it was only possible to return refs to variables that were known to be on the heap, or known to be ‘lower down’ on the stack than the callee.”

As an aside, C#7 added support for ref returns and ref locals ☺

<https://ericlippert.com/2009/05/04/the-stack-is-an-implementation-detail-part-two/>
https://unsplash.com/photos/white-brick-wall-4Zaq5xY5M_c

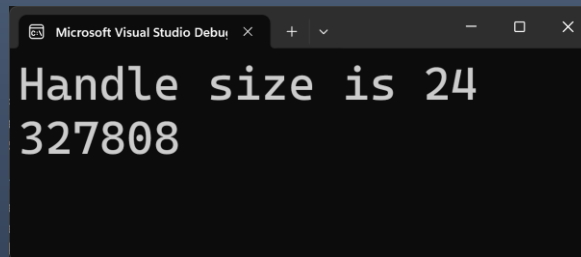
```
public sealed class Data
{
    private int value;

    public ref int GetValue()
    {
        this.value = new Random().Next();
        return ref this.value;
    }
}
```

Keep in mind you have to keep the value alive as a field value to do this, but it works.

```
unsafe
{
    var referenceDataHandleValue = typeof(ReferenceData).TypeHandle.Value;
    var referenceDataSize = Marshal.ReadInt32(referenceDataHandleValue, 4);
    Console.WriteLine($"Handle size is {referenceDataSize}");

    var referenceDataMemory = stackalloc byte[referenceDataSize];
    *(IntPtr*)referenceDataMemory = referenceDataHandleValue;
    var referenceOnStack = Unsafe.AsRef<ReferenceData>(referenceDataMemory);
    Console.WriteLine(referenceOnStack.Data);
}
```



BTW did you know you can allocate an object on the stack? It's tricky, but it can be done!

<https://steven-giesel.com/blogPost/74cc8abc-b6e3-4cb0-ba98-9c92f9a5a4d2>

Object Stack Allocation [↗](#)

This document describes work to enable object stack allocation in .NET Core.

Motivation [↗](#)

In .NET instances of reference types are allocated on the garbage-collected heap. Such allocations have performance overhead at garbage collection time. The allocator also has to ensure that the memory is fully zero-initialized. If the lifetime of an object is bounded by the lifetime of the allocating method, the allocation may be moved to the stack. The benefits of this optimization:

- The pressure on the garbage collector is reduced because the GC heap becomes smaller. The garbage collector doesn't have to be involved in allocating or deallocating these objects.
- Object field accesses may become cheaper if the compiler is able to do scalar replacement of the fields of the stack-allocated object (i.e., if the fields can be promoted).
- Some field zero-initializations may be elided by the compiler.

Object stack allocation is implemented in various Java runtimes. This optimization is more important for Java since it doesn't have value types.

GitHub issues [↗](#)

[coreclr #2104](#) Compiler should optimize "alloc temporary small object" to "alloc on stack"

There's also design/POC work into object stack allocation, which would mean in certain cases objects (reference types) would be allocated on the stack. Still is a work-in-progress as of 2023.10.23

<https://github.com/dotnet/runtime/blob/52bd95caa3563bb889548e489762388d6842dca4/docs/design/coreclr/jit/object-stack-allocation.md>
<https://github.com/dotnet/runtime/issues/11192>



So, just relax 😊. The definition doesn't really matter. While structs may be faster in some cases because of their default storage mechanism, only with performance metrics in hand should you consider changing your designs from a class to a struct or vice versa.

Myth: Gotos

Busting .NET Myths



Myth: Gotos are not allowed in C#. This is a modern language, right? And Dijkstra!!

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing

CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such disastrous effects, and I became convinced that the **go to** statement should be abolished from all "higher level" programming languages (i.e. everything except, perhaps, plain machine code). At that time I did not attach too much importance to this discovery; I now submit my considerations for publication because in very recent discussions in which the subject turned up, I have been urged to do so.

My first remark is that, although the programmer's activity ends when he has constructed a correct program, the process taking place under control of his program is the true subject matter of his activity, for it is this process that has to accomplish the desired effect; it is this process that in its dynamic behavior has to satisfy the desired specifications. Yet, once the program has been made, the "making" of the corresponding process is delegated to the machine.

My second remark is that our intellectual powers are rather geared to master static relations and that our powers to visualize processes evolving in time are relatively poorly developed. For that reason we should do (as wise programmers aware of our limitations) our utmost to shorten the conceptual gap between

dynamic progress is only characterized when we also give to which call of the procedure we refer. With the inclusion of procedures we can characterize the progress of the process via a sequence of textual indices, the length of this sequence being equal to the dynamic depth of procedure calling.

Let us now consider repetition clauses (like, **while B repeat A or repeat A until B**). Logically speaking, such clauses are now superfluous, because we can express repetition with the aid of recursive procedures. For reasons of realism I don't wish to exclude them: on the one hand, repetition clauses can be implemented quite comfortably with present day finite equipment; on the other hand, the reasoning pattern known as "induction" makes us well equipped to retain our intellectual grasp on the processes generated by repetition clauses. With the inclusion of the repetition clauses textual indices are no longer sufficient to describe the dynamic progress of the process. With each entry into a repetition clause, however, we can associate a so-called "dynamic index," inexorably counting the ordinal number of the corresponding current repetition. As repetition clauses (just as procedure calls) may be applied nestedly, we find that now the progress of the process can always be uniquely characterized by a (mixed) sequence of textual and/or dynamic indices.

The main point is that the values of these indices are outside programmer's control; they are generated (either by the write-up of his program or by the dynamic evolution of the process) whether he wishes or not. They provide independent coordinates in which to describe the progress of the process.

Why do we need such independent coordinates? The reason is—and this seems to be inherent to sequential processes—that we can interpret the value of a variable only with respect to the progress of the process. If we wish to count the number, say, of

Here's a screenshot of the original letter/paper

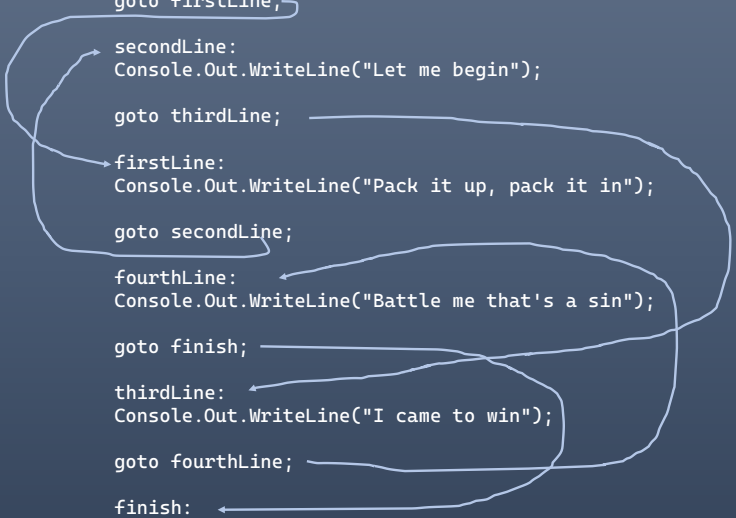
<https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

“The unbridled use of the go to statement has as an immediate consequence that it becomes terribly hard to find a meaningful set of coordinates in which to describe the process progress. ... The go to statement as it stands is just too primitive, it is too much an invitation to make a mess of one's program.” - Edsger Dijkstra

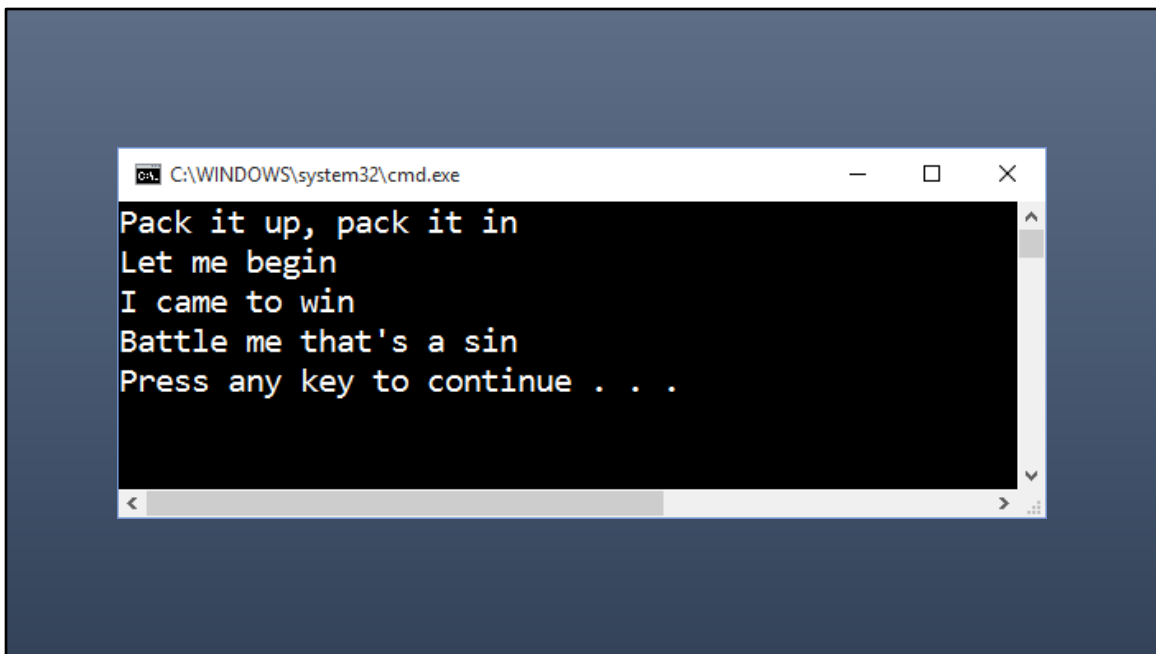
Myth: Gotos are not allowed in C#. This is a modern language, right? And Dijkstra!!

https://unsplash.com/photos/white-brick-wall-4Zaq5xY5M_c

```
private static void JumpAround()
{
    goto firstLine;
    secondLine:
    Console.WriteLine("Let me begin");
    goto thirdLine;
    firstLine:
    Console.WriteLine("Pack it up, pack it in");
    goto secondLine;
    fourthLine:
    Console.WriteLine("Battle me that's a sin");
    goto finish;
    thirdLine:
    Console.WriteLine("I came to win");
    goto fourthLine;
    finish:
    return;
}
```



The sad fact is that gotos are valid in C#.



The sad fact is that `gotos` are valid in C#.

```
private static void JumpAroundGotoFree()
{
    Console.WriteLine("Pack it up, pack it in");
    Console.WriteLine("Let me begin");
    Console.WriteLine("I came to win");
    Console.WriteLine("Battle me that's a sin");
}
```

This is so much simpler, right?

```

private static void CheckArgument(string value)
{
    if (value is null)
    {
        throw new ArgumentNullException(
            nameof(value));
    }
}

IL_0000: nop
IL_0001: ldarg.0
IL_0002: ldnull
IL_0003: ceq
IL_0005: stloc.0
IL_0006: ldloc.0
IL_0007: brfalse.s IL_0015
IL_0009: nop
IL_000a: ldstr "value"
IL_000f: newobj instance void
    [mscorlib]System.ArgumentNullException::.ctor(string)
IL_0014: throw
IL_0015: ret

```

Now, at the IL level, you can do branches (conditional or unconditional). This is fairly typical with assembly instructions or intermediate formats like IL. But it's not something that I've ever had a need to use in C#.

1 method...8 gotos

Unfortunately, I did see it used and abused once 8 years ago.

```
IPointerTypeReference pointer = typeReference as IPointerTypeReference;
if (pointer != null)
{
    typeReference = pointer.GetTargetType(context);
    bool isAssemQual = false;
    AppendSerializedTypeName(sb, typeReference, ref isAssemQual, context);
    sb.Append('*');
    goto done;
}
```

Now, just as an FYI, Microsoft uses goto in their code. You can find examples of this in Roslyn.

<https://github.com/dotnet/roslyn/blob/0cfa1529d5f8c27daf00979edf8aa8e39f77097b/src/Compilers/Core/Portable/PEWriter/TypeNameSerializer.cs#L62>

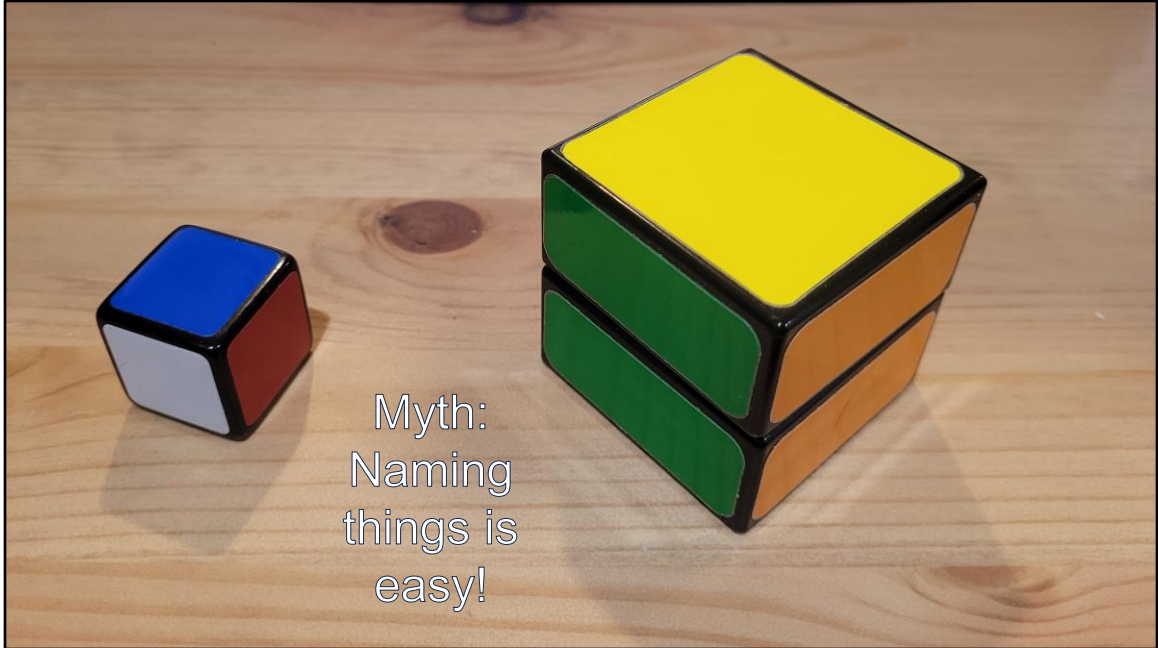
Total statement count: 922772
Total goto statement count: 115
Total goto case statement count: 453
Total goto default statement count: 158

0.081%

How many goto statements does Roslyn have? As of early 2025, not a lot

Myth: Naming Things

Busting .NET Myths



Myth: Naming things is so easy in programming!

```
public sealed class Event { ... }  
  
private readonly Event event;
```

Let's say you work on an app that creates events. If you want to use an Event class you made, using the name event doesn't work. Now you could use an underscore, but that's choosing violence, so...

```
public sealed class Event { ... }  
private readonly Event @event;
```

Or you can use the “@”. This is a verbatim identifier.

abstract	event	namespace	static
as	explicit	new	string
base	extern	null	struct
bool	false	object	switch
break	finally	operator	this
byte	fixed	out	throw
case	float	override	true
catch	for	params	try
char	foreach	private	typeof
checked	goto	protected	uint
class	if	public	ulong
const	implicit	readonly	unchecked
continue	in	ref	unsafe
decimal	int	return	ushort
default	interface	sbyte	using
delegate	internal	sealed	virtual
do	is	short	void
double	lock	sizeof	volatile
else	long	stackalloc	while
enum			

Here are all the keywords in C#. There are also contextual keywords as well.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/>


```
public class var { }  
  
public class Farmer  
{  
    private readonly Field field;  
}
```

But what if you declare something that isn't a keyword in the version of C# that you're using? This was (and still is) perfectly legal to do in C#. But in C# 3....this is disastrous because of type inference.

Using "field" as a name is currently legitimate, but in C# 14 things change

CS8981 - The type name only contains lower-cased ascii characters.

Warning wave 7

Any new keywords added for C# will be all lower-case ASCII characters. This warning ensures that none of your types conflict with future keywords. The following code produces CS8981:

C#

 Copy

```
public class lowercasename  
{  
}
```

You can address this warning by renaming the type to include at least one non-lower case ASCII character, such as an upper case character, a digit, or an underscore.

This is why this warning has been added, in a hope to limit the amount of collisions in the future.

<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/compiler-messages/warning-waves>

Myth: Concatenation

Busting .NET Myths



Myth: Self-explanatory

<https://theconversation.com/gender-stereotypes-make-teenagers-more-accepting-of-violence-33505>

```
var result = "a" + someVariable + "c";

// Or ...
var builder = new StringBuilder();
builder.Append("a");
builder.Append(someVariable);
builder.Append("c");
var result = builder.ToString();

// Or..
var result = $"a{someVariable}c";
```

Here's the question. Which technique is the "best"? There's a lot of opinions out there on the internet, so let's create our own!

Demo: Measuring Concatenation

Busting .NET Myths

Let's test the different techniques.

Myth: Parameters

Busting .NET Myths

Myth: Required parameters first, then optional parameters (if needed), then params parameter (if needed)



Myth: Parameter order has always been understood to be in a certain order. But C# can internally do things.

https://unsplash.com/photos/sliced-orange-fruit-on-black-textile-hOwcob_3dpc


```
public static void Process(  
    Guid id,  
    string content,  
    string additionalContent = "b",  
    params int[] values) { }
```

There's an order for parameters: required, optional, and params.

```
public static void Process(  
    Guid id,  
    string additionalContent = "b",  
    string content,  
    params int[] values) { }
```

```
error CS1737: Optional parameters must appear  
after all required parameters
```

If I put the optional one out of order, I'll get an error.

```
public static void Process(  
    Guid id,  
    params int[] values,  
    string content,  
    string additionalContent = "b") { }
```

```
error CS0231: A params parameter must be the  
last parameter in a parameter list
```

I'll also get an error if I move the params parameter out of order

```

        public string this[Guid id, string content,
            string additionalContent = "b"]
        {
            get => "a";
            set { }
        }

.method public hidebysig specialname instance void set_Item (
    valuetype [System.Runtime]System.Guid id,
    string content,
    [opt] string additionalContent,
    string 'value') cil managed
{
    .param [3] = "b"
    .maxstack 8

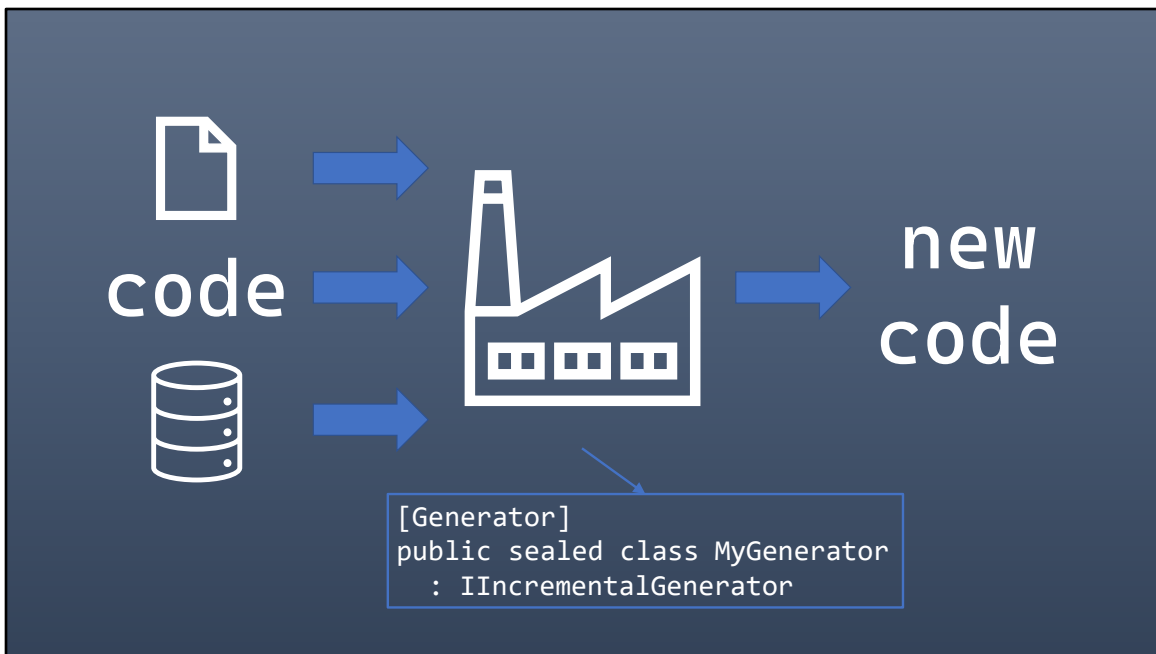
    IL_0000: nop
    IL_0001: ret
}

```

But....indexers kind of break this, because of the value parameter

```
public static void Process(  
    Guid id,  
    [Optional, DefaultValue("b")]  
    string additionalContent,  
    string content,  
    params int[] values) { }
```

But, if you use these attributes, it works for moving around optional values.



This came up when I was writing a source generator, and I assumed the parameter order was “good”. The indexer setter case I just mentioned threw a wrench into things.

Myth: Overloading

Busting .NET Myths

Myth: You can't overload methods by return type only



Myth: Most C# developers know the basics of method overloading, one of which is you can't override a method by return type only.

<https://unsplash.com/photos/selective-focus-photography-of-stop-road-sign-81QkOoPGahY>


```
public int Run(int a)
{
    return 0;
}

public Guid Run(int a)
{
    return Guid.NewGuid();
}
```

Here's a simple example of code that won't compile

```
public void Run(int a) { }  
public void Run(Guid a) { }  
public void Run(int a, string b) { }
```

“When two or more members in a type are the same kind of member (method, property, constructor, and so on) and have the same name and different parameter lists, the member is said to be overloaded.”

Let’s back up a second. Method overloading is defined this way. For simple cases, it’s not hard to figure out which method would be called.

“When two or more members in a type are the same kind of member (method, property, constructor, and so on) and have the same name and different parameter lists, the member is said to be overloaded.”

[https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229029\(v=vs.100\)](https://learn.microsoft.com/en-us/previous-versions/dotnet/netframework-4.0/ms229029(v=vs.100))

<https://github.com/dotnet/csharpstandard/blob/standard-v7/standard/expressions.md#125-member-lookup>

```
public static void CallIt<T>(object o, T t) ⇒  
    Console.WriteLine("object o, T t");  
  
public static void CallIt<T>(T t, object o) ⇒  
    Console.WriteLine("T t, object o");  
  
//Program.CallIt(new object(), new object());  
//Program.CallIt(new Program(), new Program());  
Program.CallIt(new object(), new Program());
```

For more complex scenarios, it can be difficult to figure out if it's valid, and even if it is, which method will get invoked. The first two calls won't even be allowed as they are ambiguous, but the third one works.

```
public static void CallItA<T>(object o, T t) ⇒  
    Console.WriteLine("object o, T t");  
  
public static void CallItB<T>(T t, object o) ⇒  
    Console.WriteLine("T t, object o");  
  
Program.CallItB(new object(), new object());  
Program.CallItA(new Program(), new Program());
```

A very simple way to get around this is to change the name of the method (if you can control that). Some programming languages (like Eiffel) don't support method overloading at all, which removed a fair amount of ambiguity.

7.6 Signatures and overloading

The signature of a method consists of the name of the method and the type and kind (value, reference, or output) of each of its formal parameters, considered in the order left to right. **The signature of a method specifically does not include the return type**, nor does it include the params modifier that may be specified for the right-most parameter.

Now, it's very clear according to the C# specification that the signature of a method ignores the return type. But....

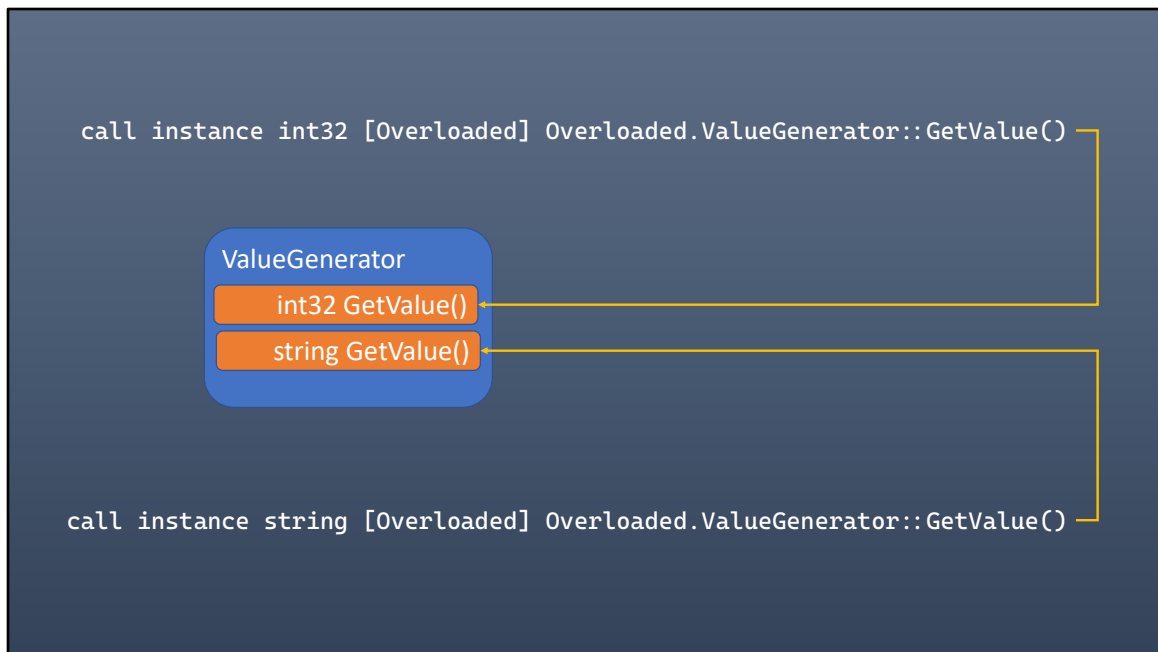
<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/basic-concepts>

https://unsplash.com/photos/white-brick-wall-4Zaq5xY5M_c

```
.method public hidebysig instance int32
  GetValue() cil managed
{
  .maxstack 8
  ldc.i4 0x64
  ret
}

.method public hidebysig instance string
  GetValue() cil managed
{
  .maxstack 8
  ldstr "100"
  ret
}
```

In IL, you can do this. When you call a method in IL, you give the return value as well, so there's no ambiguity



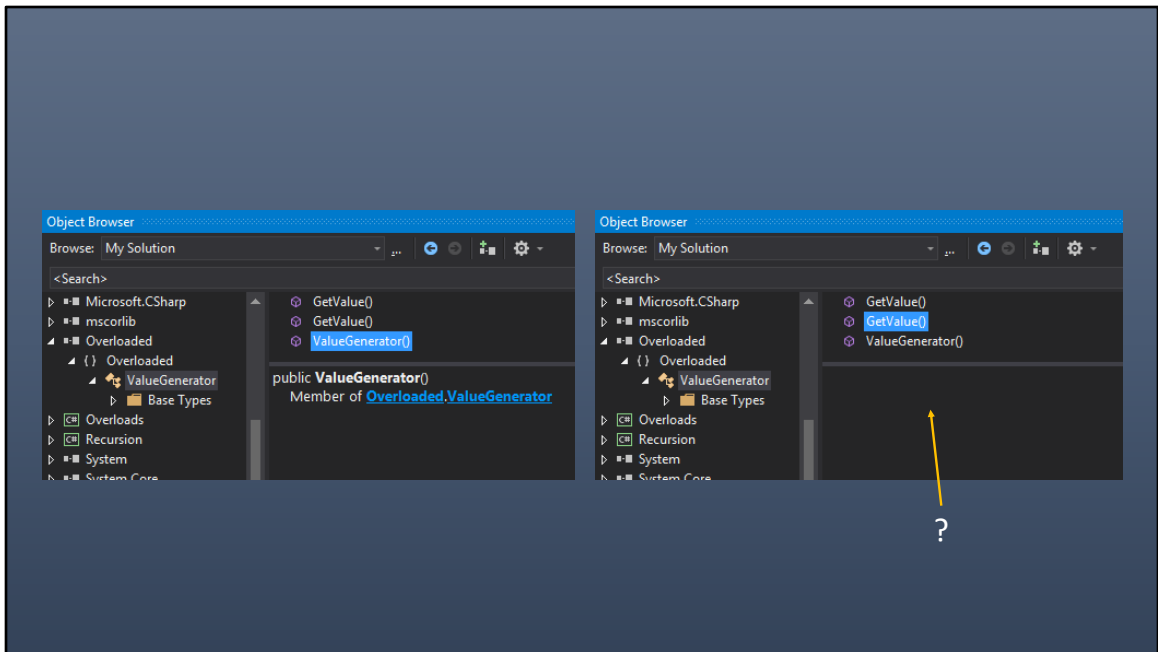
In IL, you can do this. When you call a method in IL, you give the return value as well, so there's no ambiguity

```
var generator = new ValueGenerator();

var getValueIntMethod = typeof(ValueGenerator)
    .GetMethod()
    .Single(_ => _.Name == nameof(ValueGenerator.GetValue) &&
        _.ReturnType == typeof(int));

var value = getValueIntMethod.Invoke(generator, null);
Console.WriteLine(
    $"{value}, type is {value.GetType().Name}");
```

It's possible to call these methods in C#, you just need to do some Reflection magic.



Note that the Object Browser in VS gets very confused with this setup.



This is pretty evil to do to others, so I'd advise you not to do this often ;)

<https://www.pexels.com/photo/low-angle-view-of-man-standing-at-night-316681/>

Myth: Interfaces

Busting .NET Myths



Interfaces have always been simple, but over time, they've become a lot more complicated.

Demo: Interface Evolution

Busting .NET Myths

Let's look at how interfaces have evolved.

Myth: Performance

Busting .NET Myths



Myth: .NET is slow. GC pauses, memory bloat – if you want something to run fast, forget .NET

<https://unsplash.com/photos/closeup-photo-of-orange-snail-crawling-down-pencil-on-ground-J3pkJFnpDkM>



Now, if you really want the “bare metal” experience, assembly/machine code is probably the way to go, though it’s not a guarantee of fast code, and you better know what you’re doing.

<https://unsplash.com/photos/a-heart-is-shown-on-a-computer-screen-KgLtFCgfC28>



There's always a balance. A managed language provides automatic memory collection so you don't have to worry about it, but there's a cost in having a GC present.

<https://unsplash.com/photos/brown-and-beige-weighing-scale-98MbUldcDJY>

September 10th, 2025  61 reactions

Performance Improvements in .NET 10



Stephen Toub - MSFT
Partner Software Engineer

My kids love "Frozen". They can sing every word, re-enact every scene, and provide detailed notes on the proper sparkle of Elsa's ice dress. I've seen the movie more times than I can recount, to the point where, if you've seen me do any live coding, you've probably seen my subconscious incorporate an Arendelle reference or two. After so many viewings, I began paying closer attention to the details, like how at the very beginning of the film the ice harvesters are singing a song that subtly foreshadows the story's central conflicts, the characters' journeys, and even the key to resolving the climax. I'm slightly ashamed to admit I didn't comprehend this connection until viewing number ten or so, at which point I also realized I had no idea if this ice harvesting was actually "a thing" or if it was just a clever vehicle for Disney to spin a yarn. Turns out, as I subsequently researched, it's quite real.

In the 19th century, before refrigeration, ice was an incredibly valuable commodity. Winters in the northern United States turned ponds and lakes into seasonal gold mines. The most successful operations ran with precision: workers cleared snow from the surface so the ice would grow thicker and stronger, and they scored the surface into perfect rectangles using horse-drawn plows, turning the lake into a frozen checkerboard. Once the grid was cut, teams with long saws worked to free uniform blocks weighing several hundred pounds each. These blocks were floated along channels of open water toward the shore, at which point men with poles levered the blocks up ramps and hauled them into storage. Basically, what the movie shows.

The storage itself was an art. Massive wooden ice houses, sometimes holding tens of thousands of tons, were lined with insulation, typically straw. Done well, this insulation could keep the ice solid for months, even through summer heat. Done poorly, you would open the doors to slush. And for those moving ice over long distances, typically by ship, every degree, every crack in the insulation, every extra day in transit meant more melting and more loss.

Table of contents

- Benchmarking Setup
- JIT
- Deabstraction
- Bounds Checking
- Cloning
- Inlining
- Constant Folding
- Code Layout
- [Show more](#)

Read next

Feedback - 10/10

I'd also strongly recommend reading Toub's articles, which he's done with every .NET release since 2.0. There's been a ton of perf improvements

<https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-10/>

Summary

Busting .NET Myths



In the end, remember to question yourself. The .NET, and in general programming, world is vast. Even after 14 years we can still get it wrong, and things are always in flux. Challenge your own assertions and make sure you're correct to the best of your knowledge.

Busting .NET Myths

Jason Bock

Staff Software Engineer

Rocket Mortgage

Remember...

- <https://github.com/JasonBock/BustingDotNetMyths>
- <https://github.com/JasonBock/Presentations>
- References in the notes on this slide

* Nicole Calinoiu - Back to basics: Finalization and disposition in .NET -

<https://vimeo.com/9751950>

* C# Myths - <http://binaryserialization.blogspot.com/>

* Five Myths about Managed Code -

<http://blogs.msdn.com/b/dotnet/archive/2009/10/02/five-myths-about-managed-code.aspx>

* Weak Events in C#, Take Two -

<http://www.thomaslevesque.com/2015/08/16/weak-events-in-c-take-two/>

* Dispose, Finalization, and Resource Management -

<http://joeduffyblog.com/2005/04/08/dg-update-dispose-finalization-and-resource-management/>

* Proposal: Destructable Types - <https://github.com/dotnet/roslyn/issues/161>

* Implementing IDisposable

* <https://msdn.microsoft.com/en-us/library/ms244737.aspx>

* <http://www.codeproject.com/Articles/15360/Implementing-IDisposable-and-the-Dispose-Pattern-P>

* The theory behind covariance and contravariance in C# 4 -

<http://tomasp.net/blog/variance-explained.aspx/>

* Covariance and Contravariance in Generics - <https://msdn.microsoft.com/en->

us/library/dd799517(v=vs.110).aspx

- * Generate tail call opcode -

<http://stackoverflow.com/questions/15864670/generate-tail-call-opcode>

- * Tail recursion in C# - <http://www.thomaslevesque.com/2011/09/02/tail-recursion-in-c/>

- * The Stack Is An Implementation Detail

- * Part One: <https://blogs.msdn.microsoft.com/ericlippert/2009/04/27/the-stack-is-an-implementation-detail-part-one/>

- * Part Two: <https://blogs.msdn.microsoft.com/ericlippert/2009/05/04/the-stack-is-an-implementation-detail-part-two/>

- * The Truth About Value Types -

<https://blogs.msdn.microsoft.com/ericlippert/2010/09/30/the-truth-about-value-types/>

- * StackOverflowException Class - <https://msdn.microsoft.com/en-us/library/system.stackoverflowexception.aspx>

- * ThreadAbortException Class - <https://msdn.microsoft.com/en-us/library/system.threading.threadabortexception.aspx>

- * CLR Inside Out - Handling Corrupted State Exceptions -

<https://msdn.microsoft.com/en-us/magazine/dd419661.aspx#id0070035>